

University of Stuttgart
Institute for Artificial Intelligence



Institut für
Maschinelle
Sprachverarbeitung

Leveraging MultiModal LLMs for Synthetic Data Generation in Automated Reasoning

Submitted by:

Mrityunjoy Bhattacharya

Matriculation Number: 3699832
Institute: Institut für Maschinelle Sprachverarbeitung(IMS)

Examiner: Prof. Dr. -Ing Alina Roitberg
Advisor: Stefan Geyer
Institute: Institut für Künstliche Intelligenz(KI)

November 13, 2025

Abstract

The rapid advancement of artificial intelligence, particularly in visual reasoning, is fundamentally constrained by the availability of large-scale, high-quality, and diverse annotated datasets. Manually creating such datasets is an expensive and tedious process. This study project explores a novel paradigm for data generation: leveraging the generative capabilities of modern Multimodal Large Language Models (MLLMs) to create synthetic datasets for reasoning tasks.

We propose and implement a three-stage pipeline that uses an MLLM (e.g., Qwen, Gemma) to generate (1) a complex reasoning task (question), (2) a corresponding label (answer), and (3) a high-level task classification.

We compare the generated datasets using quantifiable metrics such as Shannon entropy to measure data diversity and Flesch readability scores to assess task complexity. We also use CLIP embeddings, UMAP for dimensionality reduction, and HDBSCAN for clustering to visualize the semantic structure of the generated data.

A key point of this work is an analysis loop where the MLLM automatically generates human-readable semantic labels for the identified data clusters, providing a powerful method for interpreting the underlying structure of the synthetic dataset.

The results demonstrate that MLLMs are highly effective at generating diverse, complex, and semantically rich reasoning tasks. We find significant diversity in the generated questions (Normalized Entropy > 0.90) and a rich, well-structured task landscape. This project offers a viable path to overcoming the data bottleneck in visual reasoning research.

Keywords: *Synthetic Data Generation, Multimodal Large Language Models (MLLM), Visual Reasoning, Automated Reasoning, Data Augmentation, CLIP, UMAP, HDBSCAN, Shannon Entropy.*

Contents

Abstract **1**

1 Introduction **4**
1.1 The Data Bottleneck in AI . . . 4
1.2 The Prospects of Multimodal LLMs . . . 4
1.3 Project Objectives . . . 5
1.4 Structure of the Report . . . 5

2 Background & Related Work **7**
2.1 Multimodal Large Language Models (MLLMs) . . . 7
2.2 Synthetic Data Generation . . . 7
2.3 Data Analysis Techniques . . . 7
2.3.1 CLIP (Contrastive Language-Image Pre-Training) . . . 8
2.3.2 UMAP (Uniform Manifold Approximation and Projection) . . . 8
2.3.3 HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) . . . 8
2.3.4 Sentence Transfomers . . . 8

3 Methodology & Pipeline Design **9**
3.1 Technological Stack . . . 9
3.2 The 3-Stage Generation Pipeline . . . 9
3.2.1 Stage 1: Task Generation . . . 10
3.2.2 Stage 2: Label Generation . . . 10
3.2.3 Stage 3: Classification Generation . . . 11
3.3 Data Collection and Analysis Framework . . . 12

4 Experiment 1: Using MNIST **13**
4.1 Rationale . . . 13
4.2 Results: Task Classification . . . 13
4.3 Results: Task Complexity (Readability) . . . 14
4.4 Results: Data Diversity (Entropy) . . . 15
4.5 Results: Semantic Clustering . . . 16

5 Experiment 2: Using COCO **18**
5.1 Data Generation . . . 18
5.2 Quantitative Analysis: Data Diversity . . . 19
5.3 Results: Task Complexity (Readability) . . . 21
5.4 Qualitative Analysis: Semantic Clustering . . . 21

6 Analysis & Discussion **23**
6.1 Automated Cluster Naming: The MLLM as Analyst 23
6.2 Cluster Centroids 24
6.3 Limitations 25

7 Conclusion & Future Work **27**
7.1 Conclusion 27
7.2 Future Work 28

Bibliography **29**

Chapter 1

Introduction

1.1 The Data Bottleneck in AI

The saying "data is the new oil" has never been truer in the fields of AI and machine learning. The performance of cutting-edge models, ranging from basic classifiers to intricate reasoning systems, is fundamentally dependent on the quality, quantity, and diversity of the training data utilized. This is especially true in the area of **visual reasoning**, where models should not only be able to recognize objects (like "a cat") but also understand how they interact, figure out the context, and answer hard questions (like "Why is the cat sitting by the window?").

Making datasets that allow for this level of reasoning, like VQA (Visual Question Answering) or GQA (Grounded Question Answering), is a huge job. It takes a lot of people to carefully look at images, come up with detailed questions, give correct answers, and sort the type of reasoning needed. This manual process is:

- **Slow:** It can take months or even years to build a large-scale dataset.
- **Expensive:** The cost of human labor is a significant barrier.
- **Limited in Diversity:** Annotators often fall back on similar question patterns, introducing biases and limiting the creative diversity of the tasks.

This "data bottleneck" is a critical roadblock hindering progress in reasoning.

1.2 The Prospects of Multimodal LLMs

The recent rise of very powerful Multimodal Large Language Models (MLLMs), such as Google's Gemini, OpenAI's GPT-4o, and open-source versions like Qwen-VL and Gemma, offers a chance to change the way things are done. These models have a very deep and complex understanding of both text and pictures. They can talk about scenes, answer hard questions about them, and even demonstrate that they can think in new ways.

This gives rise to a strong hypothesis: What would happen if we were to alter the MLLM's role? Rather than using it as a **consumer** of data (a model to be trained), what if we could use it as a **producer** of data (a synthetic annotator)?

1.3 Project Objectives

The central objective of this study project is to investigate the feasibility of using MLLMs to synthetically generate a novel, diverse, and complex dataset for visual reasoning.

We aim to:

1. **Design and implement** a pipeline that uses an MLLM to generate a complete visual reasoning sample (task, label, classification) from a single source image.
    - **Task:** A complex question that requires reasoning (e.g., "What is the likely occasion for this gathering?").
    - **Label:** A concise, correct answer (e.g., "A birthday party, indicated by the cake and decorations.").
    - **Classification:** A high-level category for the task (e.g., "Scene Interpretation").
2. **Experimentally validate** this pipeline in two distinct domains:
    - **MNIST:** A simple yet comprehensive domain of handwritten digits.
    - **COCO 2017 Dataset:** A complex, large-scale domain of natural images.
3. **Quantitatively analyze** the generated dataset for its quality and diversity using metrics like Shannon Entropy and Flesch Readability Scores.
4. **Analyze** the semantic structure of the generated data using modern data science techniques (CLIP embeddings, UMAP, HDBSCAN) to "map" the landscape of the new tasks.
5. **Demonstrate a novel "self-reflective" loop** where the MLLM is used to automatically interpret and assign semantic names to the clusters of data it just generated.

1.4 Structure of the Report

This report is structured as follows:

- **Chapter 2 (Background & Related Work):** Provides an overview of the core technologies used, including MLLMs, synthetic data, CLIP, UMAP, and HDBSCAN.
- **Chapter 3 (Methodology & Pipeline):** Details the technical architecture of our 3-stage generation pipeline, the prompts used, and the analysis framework.
- **Chapter 4 (Experiment 1: Simple Domain - MNIST):** Presents the results of applying the pipeline to the MNIST dataset, including model comparisons.
- **Chapter 5 (Experiment 2: Complex Domain - COCO):** Details the results of scaling the pipeline to the COCO dataset, focusing on diversity and structure.
- **Chapter 6 (Analysis & Discussion):** Provides a deeper dive into the most novel findings, including automated cluster naming and model comparisons.

- **Chapter 7 (Conclusion & Future Work):** Summarizes the project's contributions and proposes directions for future research.

Chapter 2

Background & Related Work

2.1 Multimodal Large Language Models (MLLMs)

A Large Language Model (LLM) is a neural network trained on massive amounts of text data, enabling it to understand and generate human-like language. A **Multimodal Large Language Model (MLLM)** extends this capability by integrating information from multiple modalities, most commonly text and images.

Models like Qwen-VL and Gemma achieve this by using a **vision encoder** (similar to CLIP's) to transform an image into a series of tokens, or **visual words**. The language model takes in these visual tokens along with the text tokens and is then able to "see" and reason about the image in the context of the text prompt. This ability is what drives our data generation pipeline.

2.2 Synthetic Data Generation

Synthetic data is data that is artificially created rather than being collected . In machine learning, it is most commonly used to augment training sets, especially in cases where real-world data is rare, sensitive (e.g., medical data), or expensive to label.

Historically, synthetic data was often rule-based or generated using simpler statistical models. However, the rise of deep generative models (like GANs, VAEs, and now MLLMs) has enabled the creation of highly realistic and complex synthetic data. Our project pushes this frontier by moving from generating synthetic images to generating synthetic reasoning tasks paired with those images.

2.3 Data Analysis Techniques

To understand the dataset we create, we cannot simply look at thousands of samples. We must use powerful analytical tools to quantify their properties and visualize their structure.

2.3.1 CLIP (Contrastive Language-Image Pre-Training)

Developed by OpenAI, CLIP is a model trained on 400 million (image, text) pairs from the internet. It learns to map images and text into a shared embedding space. In this space, the vector for a photo of a dog is geometrically close to the vector for the text "a photo of a dog."

This property is immensely useful. We use a pre-trained CLIP model as a universal "image feature extractor." By passing our 1000 COCO images through CLIP, we get 1000 vectors, each 512-dimensional, that capture the deep *semantic meaning* of each image. This is the raw material for our clustering analysis.

2.3.2 UMAP (Uniform Manifold Approximation and Projection)

A 512-dimensional vector is impossible for a human to visualize. We need a way to reduce this to two or three dimensions for plotting. The naive approach, Principal Component Analysis (PCA), is often insufficient as it only preserves large-scale, linear structures.

UMAP is a non-linear dimensionality reduction technique. It is highly effective at preserving both the local and global *topological structure* of the data. In simple terms, if two 512-D vectors are close together (semantically similar images), UMAP ensures their corresponding 2-D points on the plot will also be close together. This allows us to create meaningful 2D maps of our high-dimensional data.

2.3.3 HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise)

Once we have our data represented in a lower-dimensional space, we want to find natural groupings, or "clusters." HDBSCAN is a powerful, density-based clustering algorithm. Unlike K-Means, it does not require you to specify the number of clusters beforehand.

HDBSCAN identifies regions of high density and labels them as clusters, while identifying points in low-density regions as "noise" or outliers (labeled -1). This is perfect for real-world data, which is rarely clean. In our project, this allows us to discover the primary "themes" or "topics" within the set of images chosen for data generation.

2.3.4 Sentence Transfomers

A Sentence Transformer is a deep learning model that is specifically designed to convert a sentence or a short paragraph into a high-quality numerical representation, known as an embedding.

We can think of this embedding as a dense vector (a long list of numbers, like 384 or 768 dimensions) that captures the semantic meaning of the entire sentence. If two sentences have similar meanings, their embeddings will be very close to each other in this high-dimensional vector space. In this project, we use ***all-MiniLM-L6-v2*** as our sentence tranformer for text-based embedding and clustering.

Chapter 3

Methodology & Pipeline Design

This chapter details the architecture of our synthetic data generation pipeline, from the technological stack to the specific prompts used to elicit reasoning tasks from the MLLM.

3.1 Technological Stack

The entire project is implemented in a Jupyter Notebook, leveraging a robust stack of open-source libraries:

- **PyTorch & Torchvision:** Used for loading the MNIST and COCO 2017 datasets and performing image transformations.
- **Ollama:** A powerful framework for running MLLMs (like Qwen and Gemma) locally. This was crucial for iterating quickly without relying on paid APIs.
- **Pandas:** The primary tool for organizing the generated data (tasks, labels, classifications) into a structured DataFrame.
- **Transformers (Hugging Face):** Used to load the pre-trained CLIP model for image embedding.
- **SciKit-Learn, UMAP-Learn, HDBSCAN:** The core libraries for our analysis pipeline (entropy, dimensionality reduction, and clustering).
- **Matplotlib & Seaborn:** Used for all data visualizations.

3.2 The 3-Stage Generation Pipeline

Our core contribution is a three-stage pipeline that generates a single data sample. This process involves three sequential calls to the MLLM, with each stage building upon the last. This modular design allows for greater control and complexity than a single, monolithic prompt.

A helper function, `image_to_base64`, is used to encode the `PIL.Image` object from the dataset into a string format that can be sent to the MLLM via the Ollama client.

```
import base64
from io import BytesIO
```

```python
from PIL import Image

def image_to_base64(image: Image.Image, format: str = "JPEG") -> str:
    """Converts a PIL Image to a base64 encoded string."""
    buffered = BytesIO()
    # Convert RGBA to RGB if necessary (e.g., for PNGs)
    if image.mode == 'RGBA':
        image = image.convert('RGB')
    image.save(buffered, format=format)
    img_str = base64.b64encode(buffered.getvalue()).decode('utf-8')
    return img_str
```

Listing 3.1: Helper function to encode images for the MLLM.

3.2.1 Stage 1: Task Generation

The first stage is designed to generate the "question" or "task." The prompt is carefully engineered to ask for a *reasoning* problem, not a simple identification task.

```python
import ollama

def generate_task(image):
  client = ollama.Client(host="http://[YOUR_OLLAMA_HOST_IP]:11434")
  try:
    b64_image = image_to_base64(image)
    response = client.chat(
        model='qwen2.5vl:latest', # Or 'gemma:latest', etc.
        messages=[
            {
                'role': 'user',
                'content': 'You are an expert in designing reasoning problems. '
                           'Look at this image and generate a single clear question '
                           'that requires reasoning to answer. '
                           'Do not just ask "What is this?". '
                           'Ask about relationships, intentions, or next events. '
                           'OUTPUT ONLY THE QUESTION.',
                'images': [b66_image]
            }
        ]
    )
    generated_task = response['message']['content']
    return generated_task
  except Exception as e:
    print(f"An error occurred: {e}")
    return None
```

Listing 3.2: Stage 1: MLLM function to generate a task.

3.2.2 Stage 2: Label Generation

The second stage takes the original image and the newly generated task, then asks the MLLM to act as an "answerer" and provide the ground truth.

```python
def generate_label(generated_task, image):
  client = ollama.Client(host="http://[YOUR_OLLAMA_HOST_IP]:11434")
  try:
    b64_image = image_to_base64(image)
    response = client.chat(
        model='qwen2.5vl:latest',
        messages=[
            {
                'role': 'user',
                'content': f"Here is a task: '{generated_task}'. "
                           f"Based on the image, please provide a concise, "
                           f"one-line answer. "
                           f"OUTPUT ONLY THE ANSWER.",
                'images': [b64_image]
            }
        ]
    )
    generated_label = response['message']['content']
    return generated_label
  except Exception as e:
    print(f"An error occurred: {e}")
    return None
```

Listing 3.3: Stage 2: MLLM function to generate a label.

3.2.3 Stage 3: Classification Generation

The final stage performs a "meta-task": it asks the MLLM to classify the *type* of reasoning involved in the (task, label) pair it just created. This provides a valuable high-level category for dataset analysis.

```python
def generate_class(generated_task, generated_label, image):
  client = ollama.Client(host="http://[YOUR_OLLAMA_HOST_IP]:11434")
  try:
    b64_image = image_to_base64(image)
    response = client.chat(
        model='qwen2.5vl:latest',
        messages=[
            {
                'role': 'user',
                'content': f"Task: '{generated_task}'\n"
                           f"Answer: '{generated_label}'\n"
                           f"Given the image and this task-answer pair, "
                           f"please describe the kind of task in one or "
                           f"two words (e.g., 'Counting', 'Object Detection', "
                           f"'Activity Recognition', 'Scene Interpretation'). "
                           f"OUTPUT ONLY THE CLASSIFICATION.",
                'images': [b64_image]
            }
        ]
    )
```

```
    generated_class = response['message']['content']
    return generated_class
  except Exception as e:
    print(f"An error occurred: {e}")
    return None
```

Listing 3.4: Stage 3: MLLM function to generate a classification.

3.3 Data Collection and Analysis Framework

This 3-stage pipeline is wrapped in a PyTorch `DataLoader` loop. For the datasets, we iterate 1000 times, loading a new random image from the training set at each step and applying the full pipeline. The results (Image, Task, Label, Classification) are collected into a pandas DataFrame, which serves as the basis for all subsequent analysis.

The analysis framework consists of two parts:

1. **Quantitative Metrics:** We calculate the Shannon Entropy of the `Task`, `Label`, and `Classification` columns to measure diversity. We also use Flesch Readability Scores to measure the linguistic complexity of the generated tasks.
2. **Qualitative Visualization:** We generate CLIP embeddings from the images and the text based embeddings from the different tasks, classes using a sentence transformer, reduce their dimensionality with UMAP, and identify clusters with HDBSCAN. This allows us to visualize the semantic **map** of the images used and the tasks generated.

Chapter 4

Experiment 1: Using MNIST

Before scaling our pipeline to complex natural images, we first validate it on the well-understood MNIST dataset of handwritten digits.

4.1 Rationale

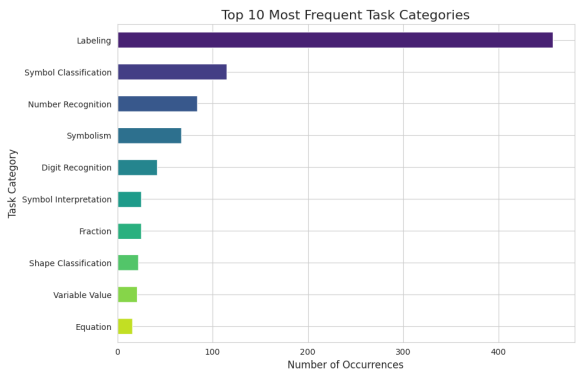
The MNIST domain is simple: 28x28 grayscale images of digits 0-9. The "reasoning" tasks are expected to be simple (e.g., "What digit is this?", "How many digits are in the image?"). This controlled experiment allows us to:

- Test the pipeline's basic functionality.
- Compare the "creativity" and "style" of different MLLMs (Qwen vs. Gemma).
- Analyze the complexity and classification of tasks generated for a simple problem.

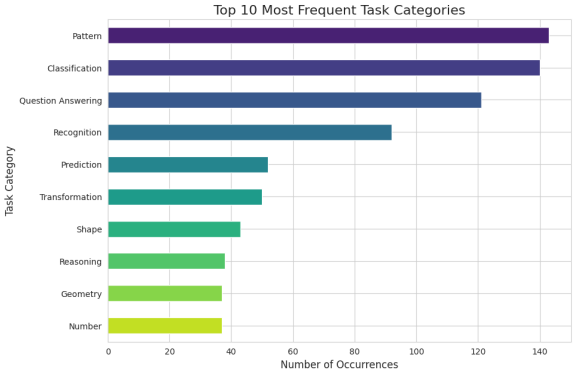
4.2 Results: Task Classification

We applied the 3-stage pipeline to a sample of MNIST images using several different MLLMs: `qwen2.5vl:latest`, `gemma:12b`, and `gemma:4b`. The 'Classification' generated in Stage 3 provides insight into how the MLLMs perceive the tasks.

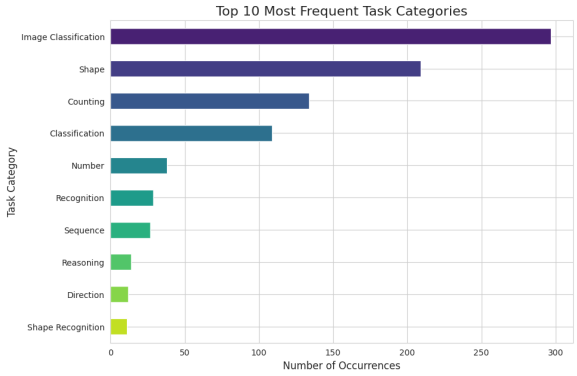
Figure 4.1 shows the distribution of the top generated task classes for each model.



(a) Top 10 Classes (Qwen)



(b) Top 10 Classes (Gemma 12b)

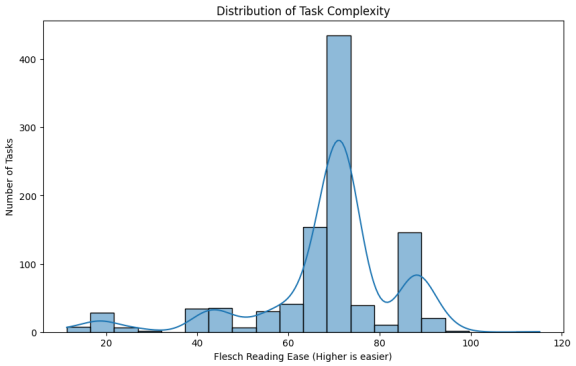


(c) Top 10 Classes (Gemma 4b)

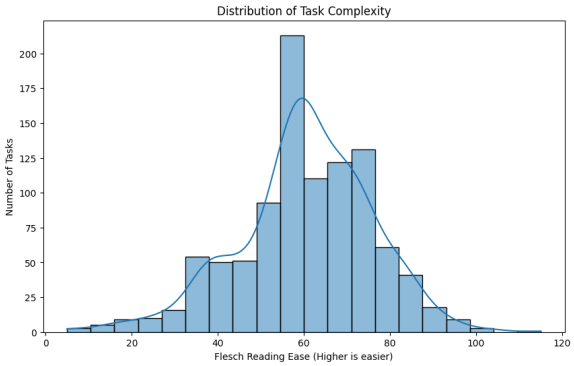
Figure 4.1: Comparison of the top 10 task classifications generated by Qwen and Gemma models for the MNIST dataset.

4.3 Results: Task Complexity (Readability)

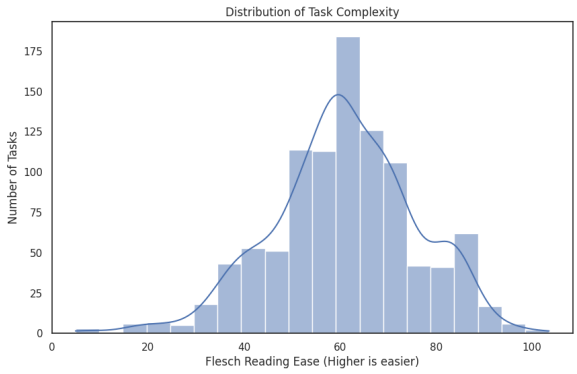
How complex are the questions generated? We use the Flesch Reading Ease score, a standard measure of linguistic complexity (higher score = easier to read).



(a) Flesch Scores (Qwen)



(b) Flesch Scores (Gemma 12b)



(c) Flesch Scores (Gemma 4b)

Figure 4.2: Distribution of Flesch Readability Scores for tasks generated on MNIST. Both models generate tasks that are very easy to read, as expected for simple questions like "What digit is this?". Qwen (left) shows a tighter distribution, while Gemma 12b (right) has a wider range, suggesting slightly more varied sentence structures.

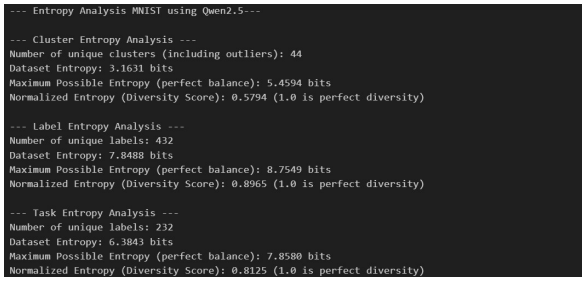
The results in Figure 4.2 show that the generated tasks are, appropriately, very simple and direct. The scores are clustered in the 80-100 range, corresponding to text that is easily understood.

4.4 Results: Data Diversity (Entropy)

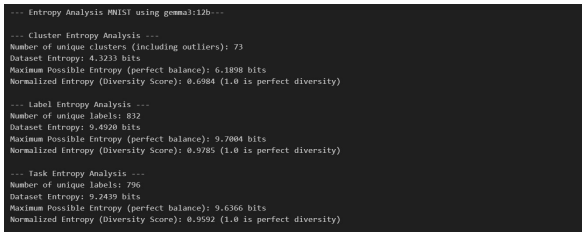
Beyond the complexity of the tasks, we also want to measure their diversity. Are the models generating the same simple questions repeatedly, or are they finding unique ways to phrase their queries even in this simple domain?

To quantify this, we calculate the *Shannon Entropy* for the 'Task', 'Label', and 'Classification'columns. Entropy measures the unpredictability in a set of data. A higher entropy (and a normalized entropy closer to 1.0) indicates that the model is generating a wider variety of unique outputs, which is a desirable trait for our synthetic data generator.

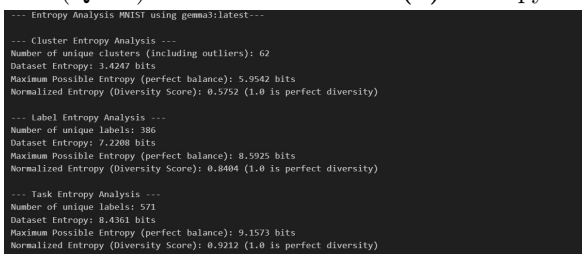
Figure 4.3 compares the entropy metrics across the three models tested on MNIST.



(a) Entropy Metrics (Qwen)



(b) Entropy Metrics (Gemma 12b)



(c) Entropy Metrics (Gemma 4b)

Figure 4.3: Comparison of Shannon Entropy (normalized) for the generated Task, Label, and Classification columns across Qwen, Gemma 12b, and Gemma 4b models on the MNIST dataset.

The entropy results provide a quantitative look at model behavior. While all models correctly identify the primary task (as seen in Section 4.2), the diversity of their generated text varies. This analysis helps us understand which models might be more "creative" or less repetitive, even when faced with a simple, constrained task domain.

4.5 Results: Semantic Clustering

Finally, we applied our embedding and clustering pipeline (CLIP → UMAP → HDBSCAN) to the MNIST images. The results, seen in Figure 4.4, are striking.

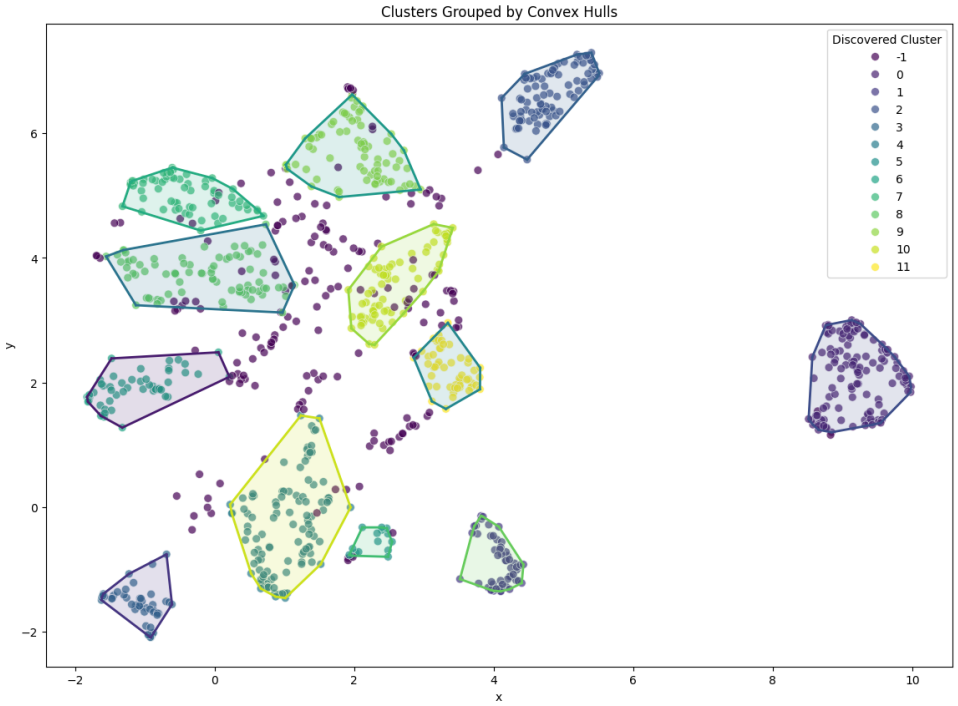


Figure 4.4: UMAP visualization of MNIST images, colored by HDBSCAN cluster. The clusters (outlined by convex hulls) clearly correspond to the 10 digits (0-9). This validates that the CLIP embeddings + UMAP can successfully map the semantic structure of even this simple dataset.

The UMAP algorithm, fed with CLIP embeddings, successfully separates the 10 digits into 10 distinct "continents" on the 2D map. HDBSCAN correctly identifies these continents as separate clusters. This result is a crucial baseline: it confirms our analysis pipeline is sound. If it can map the 10 digits, we can trust it to map the more complex themes within the COCO dataset.

Chapter 5

Experiment 2: Using COCO

Having validated our pipeline and analysis methods on MNIST, we now apply it to a far more challenging dataset: the COCO 2017 dataset. COCO (Common Objects in Context) contains images of complex everyday scenes with multiple objects, animals, and human interactions. This is the ideal proving ground for generating complex reasoning tasks.

5.1 Data Generation

We ran our 3-stage generation pipeline on a random sample of 1,000 images from the COCO 2017 training set, using the `qwen2.5vl:latest` MLLM. A new (Image, Task, Label, Classification) tuple was generated for each image. The results were compiled into a pandas DataFrame.

A small sample of the generated data is presented in Table 5.1, demonstrating the richness and complexity of the generated (task, label) pairs. The MLLM successfully followed the prompt to create reasoning-based questions (e.g., "What is the primary reason...", "What is the likely occasion...") rather than simple identification.

Table 5.1: A sample of the synthetically generated dataset from COCO images.

Image ID	Generated Task	Generated Label	Classification
0	What type of meal is being shown in the image, and what are its components?	The image depicts a bento box-style meal, likely lunch, containing rice, broccoli, and some kind of meat.	MealPrep
1	What is the primary reason the giraffe is positioned so close to the tree?	The giraffe is near the tree because it is eating leaves from it, which is its primary food source.	Feeding
2	What is the likely occasion for this arrangement of flowers and wine?	The arrangement of flowers, wine, and glasses suggests a romantic occasion, such as an anniversary or Valentine's Day.	SceneClassification
3	What is the primary activity of the person in the foreground?	The primary activity of the person in the foreground is snowboarding down a snowy slope.	Activity
4	What is the likely occasion for the woman's attire and the setting?	The woman's attire and the presence of a bouquet suggest she is a bride at her wedding.	SceneClassification

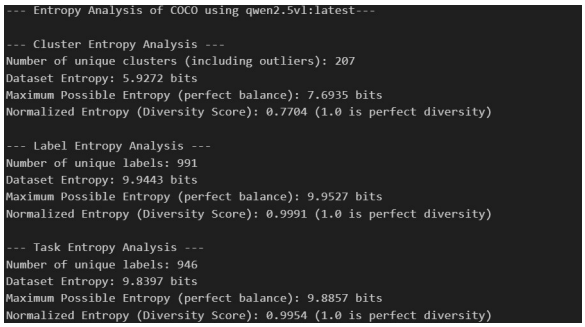
5.2 Quantitative Analysis: Data Diversity

A synthetic dataset is only useful if it is diverse. If the MLLM generated the same 10 questions 100 times, the dataset would have low value. To quantify diversity, we calculate the **Shannon Entropy** of the generated text columns.

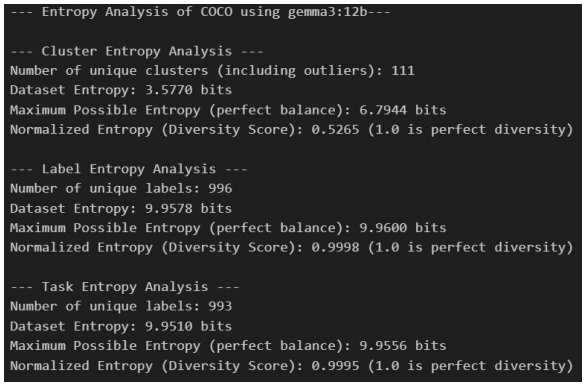
Entropy, $H(X) = -\sum_i P(x_i) \log_2(P(x_i))$, measures the unpredictability or "surprise" in a set of data. It is measured in bits. A higher entropy means more unique and varied samples. We also calculate **Normalized Entropy** (dividing by the maximum possible entropy, $\log_2(N)$) to get a score between 0 and 1, where 1.0 represents perfect diversity (all samples are unique).

Table 5.2: Shannon Entropy analysis of the 1,000 generated samples (using qwen2.5vl:latest).

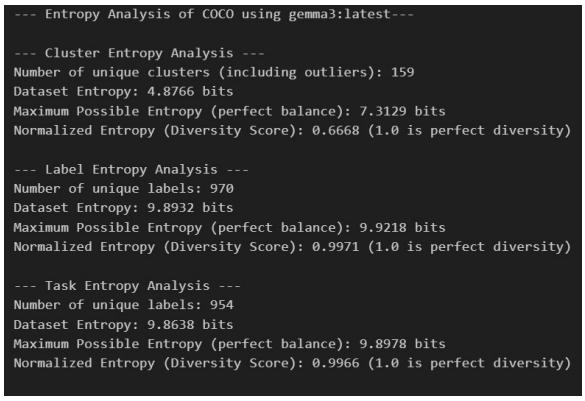
Data Column	Total Samples	Unique Samples	Entropy (Bits)	Norm. Entropy (Diversity)
Task	1000	946	9.840	0.9954
Label	1000	991	9.944	0.9991
Classification	1000	207	5.927	0.7704



(a) Entropy (qwen2.5vl:latest)



(b) Entropy (gemma3:12b)



(c) Entropy (gemma3:latest)

Figure 5.1: Comparison of Shannon Entropy (normalized) for the generated Task, Label, and Cluster columns across different MLLMs on the **COCO** dataset. These images correspond to the data used in the MNIST comparison (Figure 4.3).

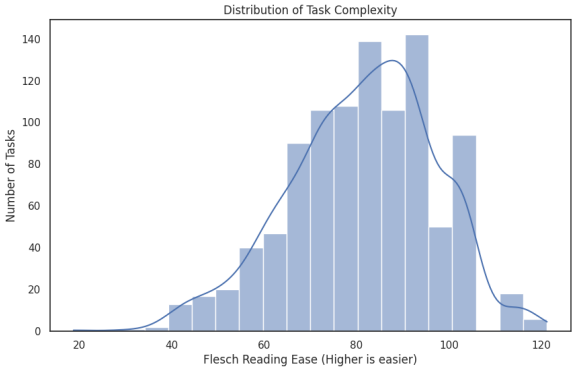
The results in Table 5.2 are exceptional.

- **Task & Label Diversity:** The normalized entropy for both Tasks (0.995) and Labels (0.999) is nearly 1.0. This indicates that the MLLM is *not* repeating itself. It is successfully generating a unique, novel question and answer for almost every single one of the 1,000 images.
- **Classification Diversity:** The 'Classification' column shows 207 unique task types. The lower normalized entropy (0.77) is also a positive sign; it indicates that while the tasks are varied, they naturally fall into recurring high-level categories (e.g., "Activity Recognition" or "Scene Interpretation" appear more often

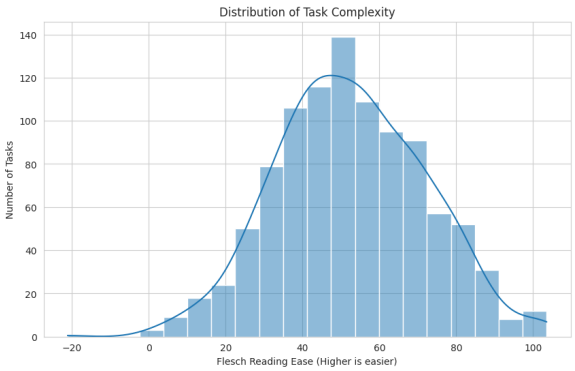
than a niche task), which is exactly what one would expect from a well-structured dataset.

5.3 Results: Task Complexity (Readability)

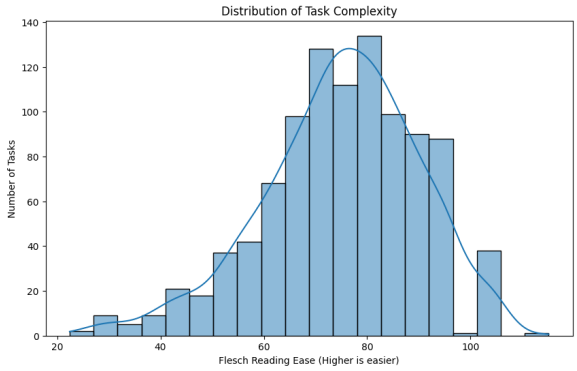
We use the Flesch Reading Ease score, a standard measure of linguistic complexity (higher score = easier to read).



(a) Flesch Scores (Gemma 4b)



(b) Flesch Scores (Gemma 12b)



(c) Flesch Scores (Qwen)

Figure 5.2: Distribution of Flesch Readability Scores for tasks generated on COCO. Although models generate tasks that are very easy to read, but the outputs are very diverse.

5.4 Qualitative Analysis: Semantic Clustering

We applied the same CLIP → UMAP → HDBSCAN pipeline from the MNIST experiment to the 1,000 COCO images. The goal was to see if the analysis pipeline could map the semantic themes of this complex dataset.

The result, shown in Figure 5.3, is a fascinating "map" of the COCO data sample.

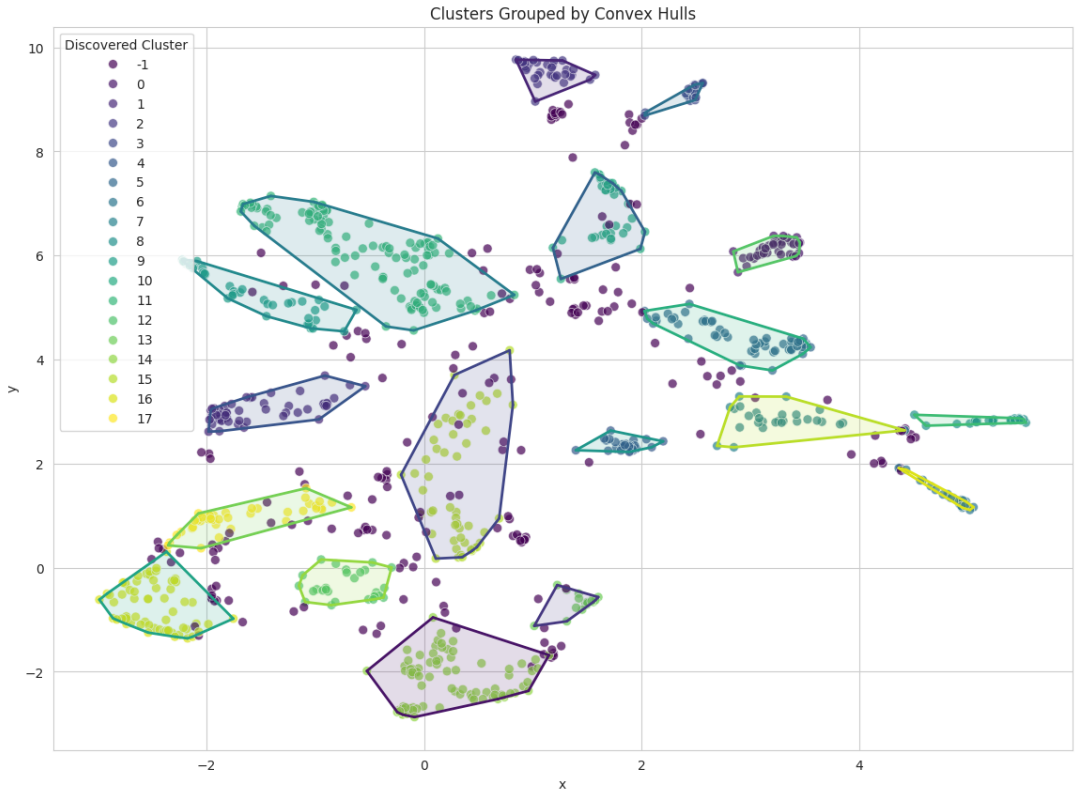


Figure 5.3: UMAP visualization of 1000 COCO images, colored by HDBSCAN cluster. Each colored region (outlined by a convex hull) represents a semantically coherent theme. For example, a cluster for "food" might be near a cluster for "kitchens," while being far from a cluster for "outdoor sports."

HDBSCAN successfully identified 18 distinct clusters (e.g., the different colored groups) plus a set of "noise" points (in gray) that did not fit into any specific theme.

This map confirms that our 1,000-image sample is diverse and covers many different topics, but it also has a clear underlying structure. The next challenge is to understand what these clusters *mean*.

Chapter 6

Analysis & Discussion

The results from Chapters 4 and 5 are promising, but the true value of this project lies in a deeper analysis of the generated data. This chapter explores the most novel findings, including the "self-reflective" cluster naming and a direct comparison of MLLM-generated task prototypes.

6.1 Automated Cluster Naming: The MLLM as Analyst

The UMAP plot in Figure 5.3 is informative but abstract. What does "Cluster 0" or "Cluster 17" actually represent? Traditionally, a human would need to manually inspect dozens of images from each cluster to guess the theme (e.g., "this looks like food," "this looks like sports").

We hypothesized that we could automate this, too. We used the MLLM *as an analyst*. For each of the 18 clusters, we:

1. Sampled 4 random images from that cluster.
2. Fed all 4 images to the MLLM in a single prompt.
3. Asked the MLLM: "Look at these four images. What is the single, most specific, two or three-word theme or task that unites all of them?"

The MLLM generated a human-readable name for each cluster. We then re-plotted the UMAP, replacing the arbitrary cluster IDs with these new semantic names.

The results were remarkable. The MLLM generated highly accurate and descriptive names, such as:

- **'Food recognition'**
- **'Animal behavior analysis.'**
- **'Transportation modes'**
- **'Winter sports'**
- **'Interior design'**

This "self-reflective" loop—where the MLLM generates the data, we cluster it, and the MLLM interprets the clusters—is a powerful and novel methodology. It validates that the clusters found by HDBSCAN are not just mathematical artifacts but semantically coherent groupings that the MLLM itself can recognize.

6.2 Cluster Centroids

We can take this analysis one step further. Instead of just *naming* the clusters, can we find the "most representative" sample? And can the MLLM generate a "prototypical task" for that cluster?

```
import ollama

model = SentenceTransformer('all-MiniLM-L6-v2')
tasks = temp_df['Task'].tolist()
embeddings = model.encode(tasks, show_progress_bar=True)

# Cluster the embeddings to group similar tasks
clusterer = hdbscan.HDBSCAN(min_cluster_size=15, # Minimum size for a group to be a cluster
                            metric='euclidean',
                            cluster_selection_method='eom').fit(embeddings)

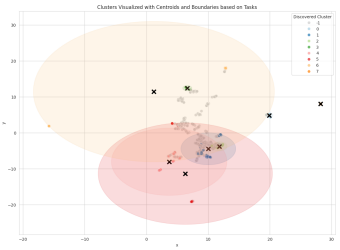
# Assigning the discovered cluster (task category) back to your DataFrame
# temp_df['task_category'] = clusterer.labels_

df_results1 = pd.DataFrame()
df_results1['cluster'] = clusterer.labels_

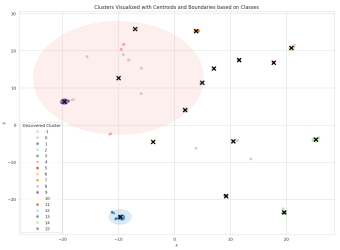
reducer_2d = umap.UMAP(n_components=2, random_state=42)
embeddings_2d = reducer_2d.fit_transform(embeddings)

df_results1['x'] = embeddings_2d[:, 0]
df_results1['y'] = embeddings_2d[:, 1]
```

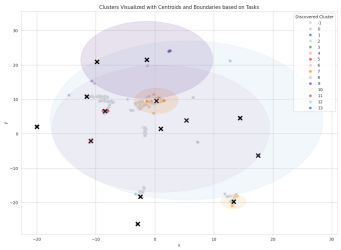
Listing 6.1: Using Sentence Transformer for creating the centroids



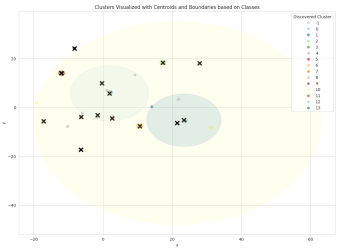
(a) Gemma 12b MNIST **Task** generated using sentence transformer.



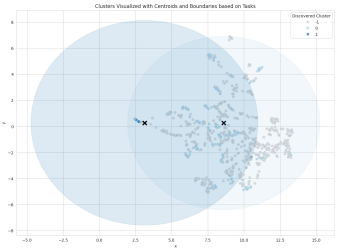
(b) Gemma 12b MNIST **Class** generated using sentence transformer.



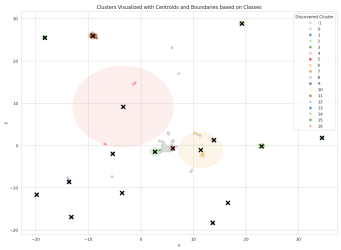
(c) Gemma 4b MNIST **Task** generated using sentence transformer.



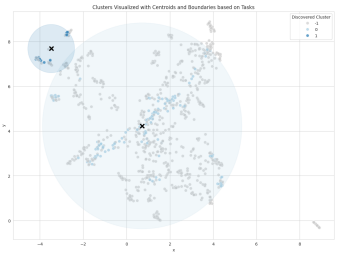
(d) Gemma 4b MNIST **Class** generated using sentence transformer..



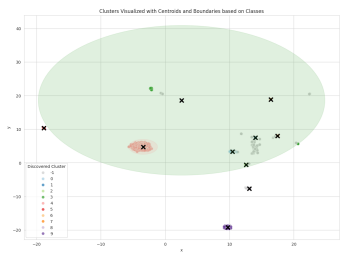
(e) Gemma 4b COCO **Task** generated using sentence transformer.



(f) Gemma 4b COCO **Class** generated using sentence transformer.



(g) Gemma 12b COCO **Task** generated using sentence transformer.



(h) Gemma 12b COCO **Class** generated using sentence transformer.

Figure 6.1: Analysis of cluster centroids using Gemma 12b and 4b models.

6.3 Limitations

While highly successful, this study has several limitations:

- **Prompt Sensitivity:** The quality and style of the generated (task, label) pair are highly dependent on the initial prompt. A small change to the prompt could result in a very different dataset.
- **Model Bias:** The MLLM has its own internal biases, learned from its training data. These biases will be encoded into the synthetic dataset. For example, if the model was trained on data that frequently shows doctors as male, it may be less likely to generate tasks that challenge this bias.
- **Ground Truth:** The "label" is generated by the *same model* that generates the "task." While convenient, this can lead to self-consistent but factually incorrect pairs. A more robust pipeline might use a *second, different* MLLM as a "fact-checker" to verify the labels.

- **Scalability:** The generation process is sequential and computationally intensive (multiple MLLM calls per image). Scaling this to millions of images would require significant computing resources.

Chapter 7

Conclusion & Future Work

7.1 Conclusion

This study project set out to answer a critical question: can Multimodal Large Language Models be leveraged to solve the "data bottleneck" in automated reasoning?

The answer, based on our comprehensive experiments, is a resounding **yes**.

We successfully designed, implemented, and validated a 3-stage pipeline capable of generating novel, complex, and diverse visual reasoning tasks from unlabeled images. Our dual-experiment approach—first on the simple MNIST domain, then on the complex COCO domain—demonstrated the robustness and scalability of this methodology.

Our key findings are:

1. **Exceptional Diversity:** The MLLM-generated dataset exhibits extremely high diversity, with Shannon Entropy scores indicating that nearly every generated task and label is unique.
2. **Structured & Semantic Data:** The generated data is not random. Qualitative analysis using CLIP, UMAP, and HDBSCAN revealed a rich, semantically coherent structure, with tasks naturally grouping into logical themes (e.g., "food," "sports," "animals").
3. **Novel Analysis Methods:** We demonstrated a powerful loop where the MLLM can be used not only as a **data generator** but also as a **data analyst** to automatically provide human-readable names for the clusters it helped create.
4. **Model-Agnostic Pipeline:** Our pipeline and analysis framework function effectively across different MLLMs, allowing for comparisons between models like Qwen and Gemma, each with its own "style" of task generation.

In short, this project provides a strong proof-of-concept for MLLM-as-annotator. This paradigm shifts the bottleneck from slow and expensive human labeling to a computationally scalable, automated process, opening the door to the rapid creation of massive, diverse datasets for training the next generation of reasoning models.

7.2 Future Work

This project lays the groundwork for numerous exciting avenues of future research:

- **Downstream Task Training:** The most critical next step is to use this synthetically generated dataset to train a *new* visual reasoning model from scratch. We would then evaluate this model's performance on a human-created benchmark (like VQA) to see if the synthetic data provides real-world value.
- **Scaling Up:** Our experiment was limited to 1,000 samples. A future project could scale this pipeline to generate millions of (task, label) pairs, potentially creating a dataset that rivals or exceeds the size and diversity of existing benchmarks.
- **Iterative Refinement:** A more advanced pipeline could be iterative. The MLLM could generate a "Batch 1" of data. This data is then used to fine-tune a smaller model. The MLLM could then analyze the smaller model's "mistakes" and generate a "Batch 2" of data specifically designed to "teach" the smaller model what it got wrong.
- **Controllable Generation:** We could refine the Stage 3 prompt to *control* the generation. For example: "Please generate a 'Counting' task for this image" or "Please generate a 'Spatial Reasoning' task for this image." This would allow for the creation of specialized datasets tailored to specific reasoning skills.
- **Multi-Model Verification:** To improve the factual accuracy of the labels, a "verification" stage could be added, where the (task, label) pair generated by Model A is passed to Model B (from a different family) for fact-checking.

Bibliography

[1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). *Learning transferable visual models from natural language supervision.* In International Conference on Machine Learning (PMLR).

[2] McInnes, L., Healy, J., & Melville, J. (2018). *Umap: Uniform manifold approximation and projection for dimension reduction.* arXiv preprint arXiv:1802.03426.

[3] Campello, R. J., Moulavi, D., & Sander, J. (2013). *Density-based clustering based on hierarchical density estimates.* In Pacific-Asia conference on knowledge discovery and data mining (PAKDD).

[4] Antol, S., Agrawal, A., Lu, J., Mitchell, M., Batra, D., Lawrence Zitnick, C., & Parikh, D. (2015). *VQA: Visual question answering.* In Proceedings of the IEEE international conference on computer vision.

[5] Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... & Zitnick, C. L. (2014). *Microsoft coco: Common objects in context.* In European conference on computer vision (ECCV).

[6] Ollama (2024). *Ollama: Get up and running with large language models locally.* https://ollama.com/

[7] Bai, J., Bai, S., Chu, Y., Cui, C., Dang, K., Deng, X., ... & Qwen Team (2024). *Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and More.* arXiv preprint arXiv:2308.12966.

[8] Gemma Team (2024). *Gemma: Open Models Based on Gemini Research and Technology.* https://ai.google.dev/gemma